

Пользовательские расширения конвертера БД

Как известно, обновление структуры схем баз данных компании производится в автоматизированном режиме с помощью специальной утилиты "Конвертер БД". Основная задача конвертации - загрузить состояние объектов из схемы конвертируемой БД, извлечь целевое состояние объектов из метаданных проекта, описанных пользователем в справочниках [dicx](#), и построить по ним план конвертации - последовательность SQL-инструкций для преобразования текущего состояния БД к требуемому.

Иногда возникают ситуации, когда план конвертации содержит нежелательные инструкции или приводит к слишком долгой конвертации. Для решения данной проблемы был разработан механизм пользовательских расширений конвертации.



Конвертер или Расширение конвертации?

Исторически сложилось, что пользовательские расширения называют "конвертерами". Данный термин приводит к путанице - непонятно, о чем идет речь "Конвертере БД" или "Пользовательском расширении конвертации". Вместо него лучше использовать термин "Расширение конвертации".

Основное назначение данного механизма - обеспечение интерфейса доступа к плану конвертации схемы для внесения в него необходимых изменений посредством создания обработчиков или декларативных действий.



В какой момент конвертации выполняются расширения?

Необходимо помнить, что пользовательские расширения, кроме декларативных, выполняются уже после построения плана конвертации, непосредственно в процессе его выполнения. Данный факт необходимо учитывать при написании логики расширения.

Регистрация выполнения

Все расширения конвертации выполняются **посхемно** и только **один раз**. В случае обычного сервиса - схема `public`, в случае мультиарендного - пользовательские схемы `_XXXXXXXX`. После завершения конвертации имя расширения регистрируется в служебной таблице соответствующей схемы и при следующем запуске не будет выполнено, так как считается, что все изменения для плана конвертации успешно применены.



Как проверить выполнено ли мое расширение в этой БД?

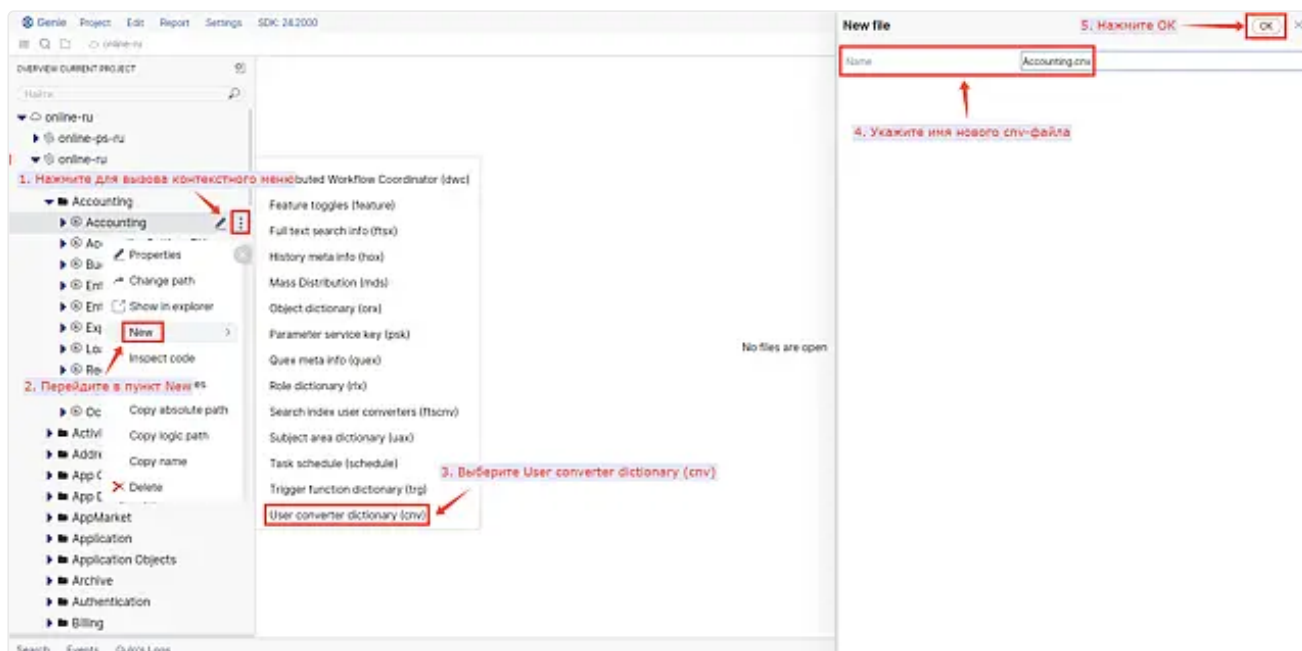
Проверка факта выполнения расширения в БД осуществляется запросом к таблице `public."ВыполненноеРасширениеОсновная"` для обычного сервиса или `"<пользовательская схема>."ВыполненноеРасширение"` для мультиарендного с фильтром по полю "Название".

Если после конвертации выяснилось, что расширение требует коррекции и повторного запуска своей "второй версии", то необходимо изменить имя расширения, например, `"MyExtension_v2"`. Основная идея - создать новое имя, которое не зарегистрировано в служебной таблице.

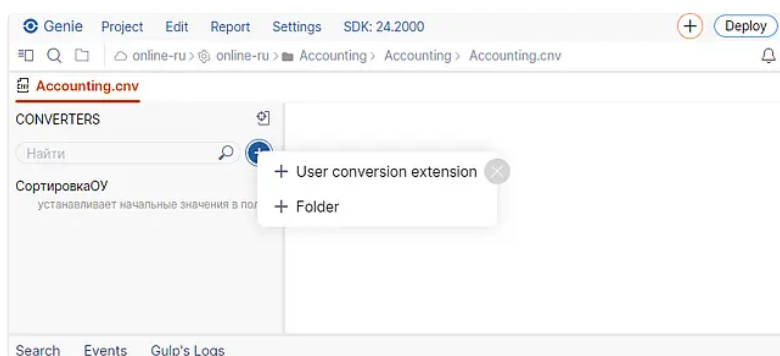
Справочник пользовательских расширений (*.cnv)

Справочник расширений конвертера - стандартный СБИС-контейнер метаданных, который содержит описания набора пользовательских расширений. В файловой системе сохраняется в виде XML-файла с расширением `.cnv`.

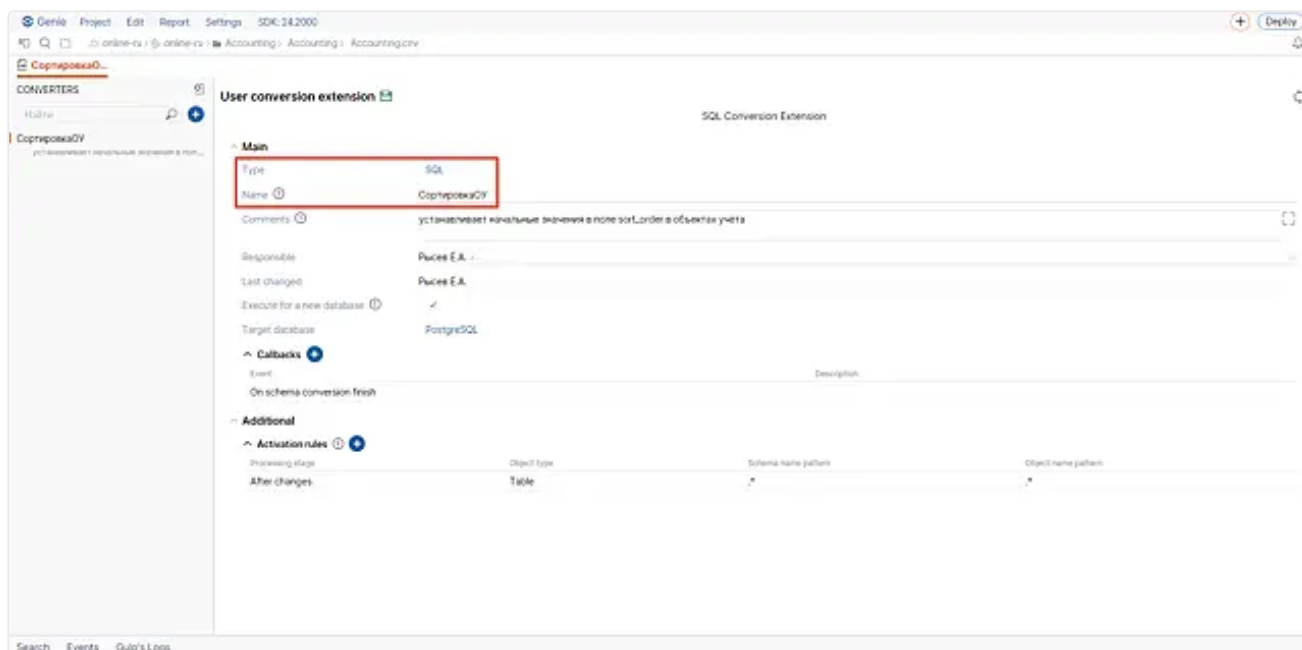
Для создания справочника необходимо в [Genie](#) найти целевой БЛ-модуль в дереве проекта и в контекстном меню выбрать пункт **New** → **User converter dictionary (cnv)**.



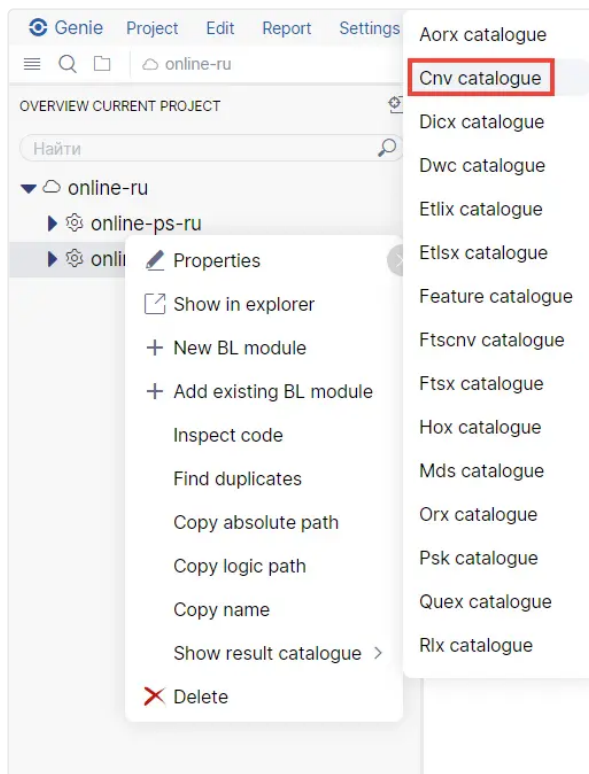
В справочнике новые пользовательские расширения создаются нажатием кнопки "+". В контекстном меню выбирается создание расширения или отдельной папки, в которой затем будут созданы расширения.



Затем необходимо выбрать тип расширения и задать имя:



В рамках одного БЛ-модуля можно создать несколько файлов данного типа. При конвертации содержимое всех справочников .cnv из всех БЛ-модулей конвертируемых сервисов компонуется в общий справочник, который содержит множество описаний пользовательских расширений. Если в процессе компоновки будут обнаружены расширения с одинаковыми именами, то будет взято встреченное последним (порядок компоновки определяется порядком загрузки модулей БЛ). Для просмотра обобщенного справочника расширений конвертации необходимо выбрать пункт **Show result catalogue → Cnv catalogue** в контекстном меню свойств сервиса:



Типы пользовательских расширений

На данный момент в Wasaby Framework поддерживаются следующие типы расширений:

- **SQL** - устаревший тип расширений, логика обработчиков реализуется на языке SQL и выполняется непосредственно сервером БД. В случае с СУБД PostgreSQL используется расширение PL/PGSQL. SQL-расширения безальтернативны в связке "offline-приложение + PostgreSQL".
- **Python** - рекомендуемый к использованию тип расширений на сервере. Для offline и мобильных приложений использоваться не может.
- **Binary** - разработаны для встраиваемых проектов, использующих СУБД SQLite для хранения данных, например, мобильные приложения. Логика обработчиков описывается непосредственно в БЛ-модуле на C++.
- **Declarative** - специальный тип расширений, который позволяет описать изменения в проекте с помощью фиксированного набора типовых операций без написания кода.

Структура пользовательского расширения

Общие поля

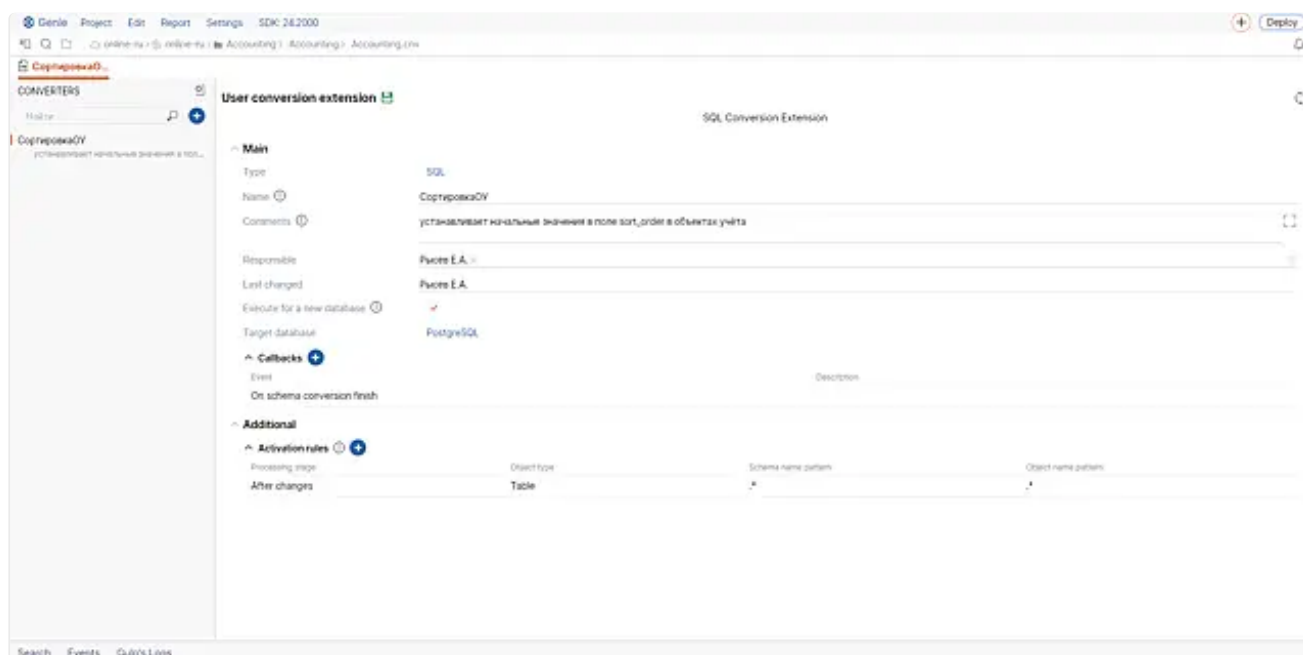
Базовый диалог редактирования пользовательского расширения начинается с заголовка в котором отображается тип текущего расширения. Далее следует блок с общими параметрами (секция Main) набор которых может меняться в зависимости от типа расширения:

- Обязательные параметры для всех расширений:
 - **Name** - уникальное имя пользовательского расширения
 - **Responsible** - информация об ответственном за расширение
 - **Last changed** - информация о последнем изменившем расширение
 - **Comments** - обязательная к заполнению графа с подробными комментариями о логике расширения
- Опциональные параметры:
 - **Execute for new database** - выполнять расширение и для новой и для существующей БД. Если флаг не выбран, то расширение будет выполнено только для существующей БД.
 - **Execute for schema** - тип схемы, на которой разрешено выполнение расширения:
 - **Personal** - выполнять на всех клиентских схемах,
 - **Public** - выполнить только для public-схемы,

- **Any** - выполнять на всех схемах.
- **Target database** - тип СУБД, для которой необходимо выполнить данное расширение. Вариант Any выполнит расширение конвертации для любого типа СУБД: PostgreSQL и SQLite.
- **Stage** - фаза конвертации, на которой необходимо запускать расширение.
 - **Preliminary phase** - подготовительная,
 - **Main phase** - основная,
 - **Final phase** - заключительная.
- **Disable transaction** - флаг, выключающий создание транзакции при вызове обработчиков расширения. Доступен только при заключительной фазе конвертации. Предполагается, что код расширения сам создаст транзакцию через `sbis.CreateTransaction()`. Данная функциональность может потребоваться например для пост-обработки больших объемов данных короткими транзакциями.

После основных параметров следует опциональный блок с дополнительными параметрами (секция **Additional**):

- **Callbacks** - список обработчиков расширения
- **Declarative ...** - списки декларативных действий



Информация об авторе

Крайне важно заполнить точную информацию об авторе, чтобы, в случае возникновения проблем с данным расширением в процессе конвертации, можно было быстро найти ответственного



Объединение обработчиков основной и предварительной фазы

Если конвертация была запущена на основной фазе, но при этом имеются расширения конвертации, которые не зарегистрированы как выполненные и должны быть выполнены на предварительной фазе, то они будут запущены на основной.

Обработчик события



Данный раздел содержит общее описание концепции "обработчика события", для подробностей необходимо перейти к описанию конкретного типа расширений.

Обработчики событий присутствуют в Python, SQL и Binary расширениях. Список обработчиков - основной интерфейс для внесения корректировок в план конвертации для Python и SQL-расширений. Binary-расширения регистрируют обработчики в C++ коде.

Обработчик события - функция обратного вызова, которая будет вызвана в процессе выполнения плана конвертации "Конвертером БД". Все обработчики вызываются в **одной транзакции** выполнения плана конвертации. Перед вызовом **путь поиска** объектов БД инициализируется **текущей конвертируемой схемой**.

Каждый обработчик регистрируется на наступление определенного события конвертации:

- **On schema conversion start** - вызывается перед выполнением плана конвертации на схеме.
- **On schema conversion finish** - вызывается после выполнения плана конвертации на схеме.
- **On [create/drop/alter] [table/column/...]** - обработчики, вызываемые при исполнении DDL-инструкции из плана конвертации. Например, до или после создания индекса.

При наступлении события плана конвертации будут последовательно вызваны обработчики **всех** расширений, которые зарегистрировали обработчик на соответствующее событие.



Исключение в обработчике

Если обработчик "бросает" исключение, конвертация схемы прерывается, все изменения, осуществленные в процессе конвертации, откатываются.

Для обработчиков DDL-инструкций можно дополнительно настроить момент вызова - "до" или "после" выполнения DDL-инструкции. Для Python-расширений момент вызова настраивается непосредственно в редакторе обработчика, для SQL-расширений момент срабатывания задается в отдельном списке настроек **Activation rules**.



Уникальность обработчиков

Список обработчиков не может включать в себя более одного обработчика на одно и то же событие. Например, расширение не может содержать два обработчика на событие Create table. Для Python-расширений учитывается момент срабатывания, то есть допустимо создать обработчик на "до" изменений и "после" изменений для одного и того же события.

Отмена запланированной инструкции

Выполнение DDL-инструкции можно отменить в случае, если обработчик вызывается **до изменений**. Блокировка предполагает, что пользователь сам выполнит необходимые преобразования позже в другом обработчике в рамках конвертации. Чтобы отменить выполнение в Python-расширении обработчик должен вернуть True, в SQL-расширении обработчик должен записать true в переменную result. В результате запланированный SQL-скрипт инструкции не будет выполнен и алгоритм конвертации перейдет к следующей инструкции в плане, при этом обработчик "после" изменений не будет вызван.



Блокировка действует только на текущую конвертацию

Основной шаблон действий при блокировке выполнения инструкции: заблокировать запланированные изменения над объектом БД и выполнить собственные изменения в обработчике **On schema conversion finish**. Если действие над объектом было заблокировано и **не обработано** в дальнейшем, при этом в метаданных проектах заблокированный объект остался, то при следующей конвертации объект будет создан в БД.

Контекст инструкции обработчика

Контекст инструкции - структура данных, содержащая мета-информацию об инструкции в виде пар строк ключ-значение.

Контекст позволяет обрабатывающей логике понять, о какой конкретно инструкции идет речь и принять решение о дальнейших действиях.

Список ключей контекста:

- Присутствуют всегда:
 - stage - этап обработки, множество значений: before ("до") и after ("после").
 - schema - имя конвертируемой схемы.
 - cnv_type - тип конвертации. Возможные значения:
 - lite - легкая,
 - full - полная,
 - per_schema - посхемная.
- Зависит от типа инструкции:
 - table - имя таблицы, также присутствует для дочерних элементов.
 - column - имя обрабатываемого столбца таблицы.
 - type - тип обрабатываемого столбца таблицы.

- trigger - имя обрабатываемого триггера.
- index - имя обрабатываемого индекса.
- constraint - имя обрабатываемого ограничения целостности.
- sequence - имя обрабатываемой последовательности.
- function - имя обрабатываемой хранимой процедуры.
- arguments - строка с аргументами обрабатываемой хранимой процедуры.
- num_arguments - число аргументов обрабатываемой хранимой процедуры.



Имена объектов в контексте

Имя объекта в контексте всегда инициализируется именем, которое будет у объекта в БД. Скорее всего оно не будет совпадать с именем объекта из метаданных проекта, так как имя может быть укорочено и/или транслитерировано для удовлетворения требований к идентификаторам в БД. Зависимым объектам в имя добавляются служебные суффиксы и префиксы для обеспечения уникальности идентификатора.

Существует возможность создавать пользовательские пары ключ-значение в контексте для сохранения и передачи состояния выполнения между обработчиками расширения. Созданные записи доступны только обработчикам данного расширения и только в текущем потоке конвертации схемы.

Ограничения для обработчиков

Необходимо помнить, что пользовательские расширения конвертации работают в контексте работы основного алгоритма, поэтому на код обработчиков накладывается ряд ограничений для обеспечения детерминированности работы конвертера БД:

- Категорически недопустимо использовать любые внешние источники данных - подключаться к другим базам данных, вызывать методы БЛ и т. д. Возможные проблемы: состояние гонки, отсутствие доступа к домену/хосту с конвертирующего хоста, нестабильность сети, побочные эффекты приводящие к случайным неповторяющимся ошибкам, дыры в безопасности, потеря данных и т. д. Все действия должны зависеть только от текущей конвертируемой схемы БД и контекста инструкции.
- Запрещено изменять глобальную конфигурацию БД, в особенности search_path.
- Запрещено "помогать" конвертеру - по ходу конвертации модифицировать объекты, которые описаны в метаданных проекта и действия над которыми не заблокированы ни одним из обработчиков расширения.



Обработчик зависит только от контекста инструкции и состояния конвертируемой схемы

Если потребовалось что либо из запрещенного, то расширения конвертации - не тот механизм, необходимо использовать [скрипты "Хоттабыча"](#).

В обработчиках допускается:

- Создавать и модифицировать объекты БД, которые **не описаны** в метаданных проекта и **не конфликтуют** с объектами из проекта.
- Изменять данные в таблицах.
- Выполнять DDL-инструкции на объектах БД, которые были **заблокированы** в обработчиках инструкций **до** применения изменений.
- Добавлять метаданные к объектам БД.



Review

Все расширения конвертации должны в обязательном порядке проходить review кода на наличие запрещенных действий.

Пользовательское расширение конвертации на SQL

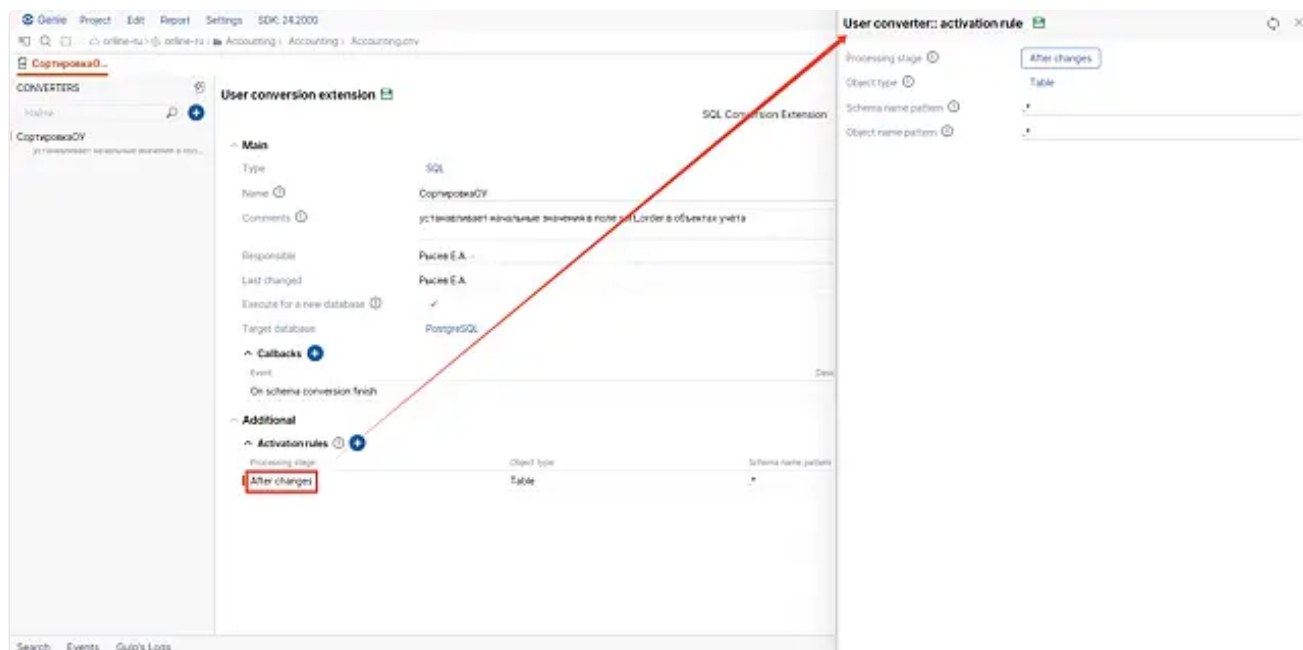


Фаза конвертации

Все SQL-расширения выполняются только на основной фазе конвертации, данное поведение изменить нельзя.

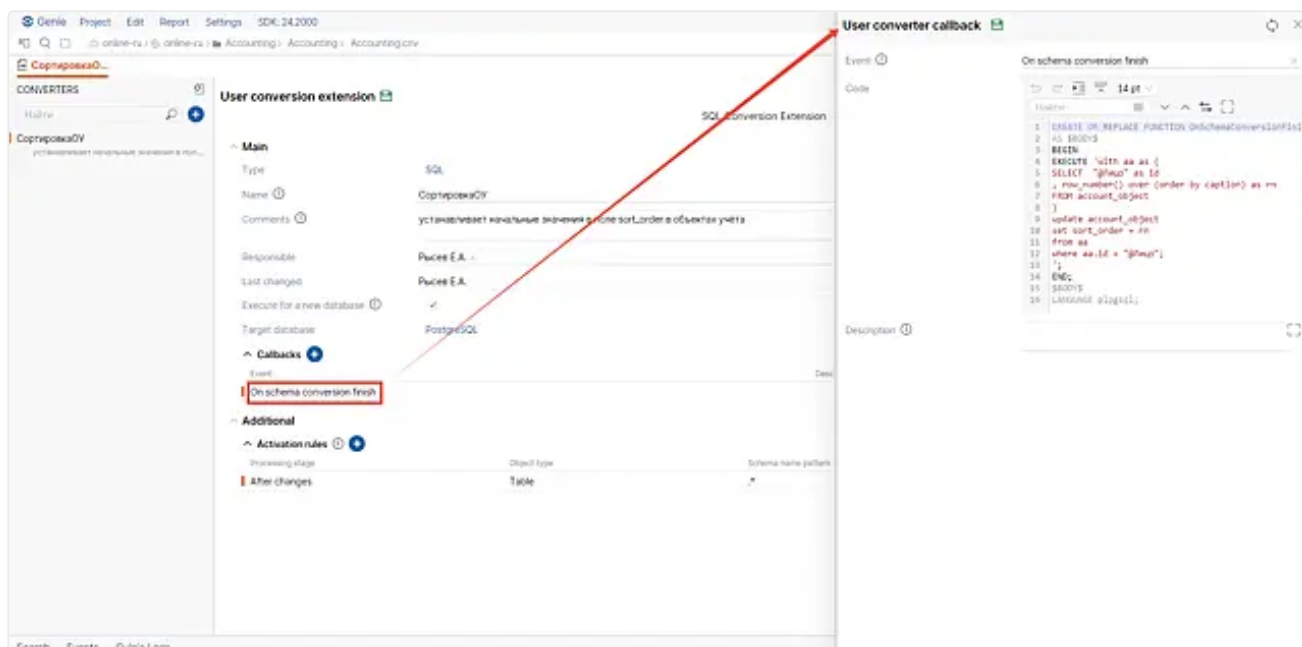
Дополнительные параметры:

- **Activation rules** - правила активации SQL-обработчиков. Для того чтобы обработчик на событие был вызван необходимо, чтобы все предикаты одного из правил успешно прошли проверку на контексте инструкции. Параметры правила активации:
 - **Object type** - тип объекта, для которого срабатывает правило активации:
 - **Table** - таблица и все табличные объекты
 - **Sequence**
 - **Any** - любой объект
 - **Processing stage** - момент обработки инструкции
 - **Before changes** - до выполнения SQL-скрипта DDL-инструкции
 - **After changes** - после выполнения SQL-скрипта DDL-инструкции
 - **Always** - вызвать обработчик и до и после изменений
 - **Schema name pattern** - условие на основе регулярного выражения, которое определяет имена схем, для объектов которых срабатывает правило активации. Например, если вы хотите выполнить пользовательское расширение для любых схем, то запишите в поле значение `.*`. Если вам нужна конкретная схема, впишите в поле её название.
 - **Object name pattern** - условие на основе регулярного выражения, которое определяет имена объектов БД, на которые срабатывает правило активации. Например, если вы хотите выполнить пользовательское расширение для всех объектов, то запишите в поле значение `.*`. Если вам нужен конкретный объект, впишите в поле его название. Подробную информацию о регулярных выражениях, а также их примеры можно посмотреть [здесь](#) или в аналогичной статье [Википедии](#).



Обработчик SQL-расширения

- **Event** - выбор типа события обработчика. Момент вызова и другие фильтры задаются в правилах активации.
- **Code** - код обработчика на SQL.
- **Description** - комментарий к логике работы обработчика.



Параметры в сигнатуре обработчика:

- context – контекст инструкции с типом hstore.
- result – статус обработки инструкции. Тип bool, по умолчанию false. Если присвоить true, то инструкция считается обработанной и будет пропущена (в случае вызова обработчика **до изменений**).

Пример обработчика на SQL

В случае, если расширение создается для СУБД PostgreSQL, поле Code имеет следующую структуру:

```

1 declare
2     -- опциональная секция с локальными переменными
3 begin
4     -- основной код находится здесь
5
6     -- получение данных из контекста
7     if context->'schema' = 'public' and context->'table' = 'my_table'
8 then
9         -- действия для my_table
10
11         -- пропустить инструкцию для my_table
12         result := true;
13
14         -- сохранение пользовательского состояния в контексте
15         context := context || hstore('my_state_flag', true);
16     end if;
17 end;
```

Технически обработчики реализованы в виде пар схема-функция. Конвертер БД автоматически создает схемы с именами всех расширений, которые необходимо выполнить в процессе конвертации. Затем в схеме с именем расширения разворачиваются функции-обработчики с именами событий на которые они должны вызываться. По завершению конвертации созданные схемы будут удалены.



Странные схемы в БД

В случае аварийного завершения конвертера схемы SQL-расширений не будут удалены, так как они создаются вне транзакции. При последующих запусках конвертации они должны пропасть.

Поскольку СУБД SQLite не поддерживает расширений SQL и каких либо хранимых процедур, код в поле Code крайне ограничен. По сути каждый обработчик – обычный SQL-скрипт без ветвлений и циклов, который вызывается в потоке

конвертации схемы. SQL-инструкции в скрипте разделяются символом `;`. Для событий над объектами БД допускается пропуск инструкции включением в конец обработчика инструкции вида:

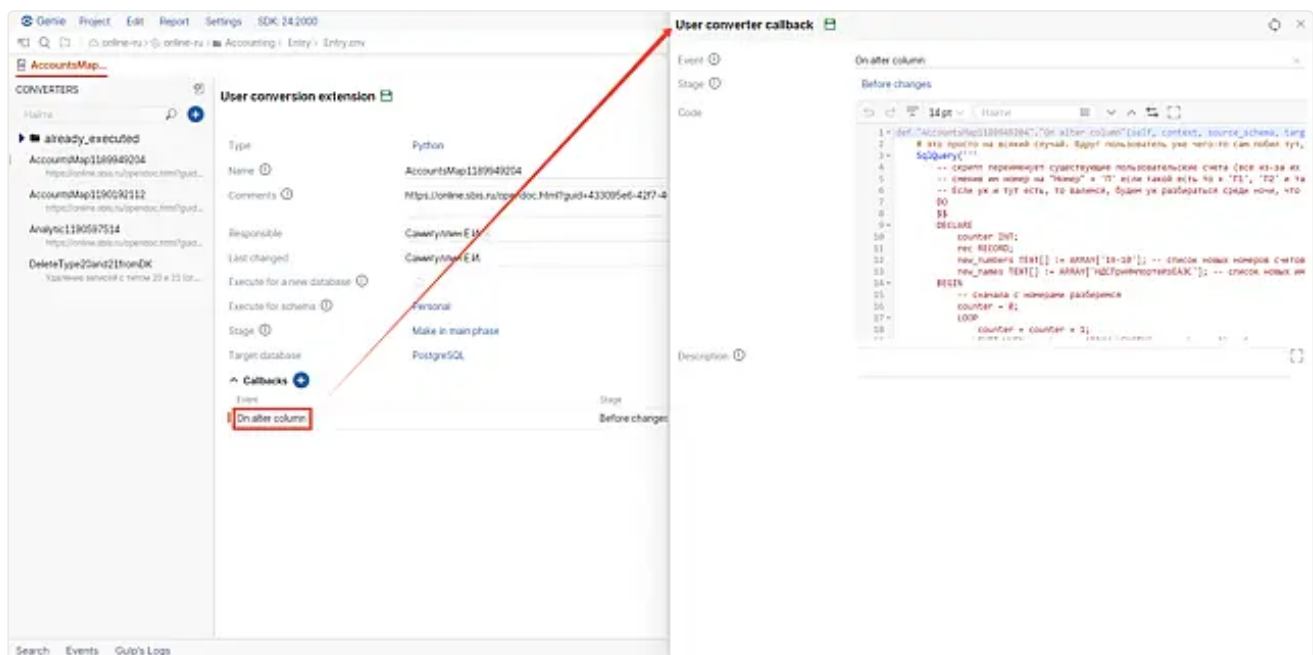
```
select exists(select 1 from context where context."table" = 'my_table');
```

Пользовательское расширение конвертации на Python

В целях обеспечения кроссплатформенности конвертера БД (зависимость от Python-интерпретатора не приемлема для некоторых продуктов), весь функционал Python-расширений был вынесен в отдельный разделяемый модуль с именем `sbis-db-converter-pyext-plugin`. Наличие модуля - опционально. Если модуль отсутствует и его не удастся загрузить в память процесса, то расширения данного типа не будут выполнены, конвертация будет продолжена без ошибок.

Обработчик Python-расширения

- **Event** - выбор типа события обработчика.
- **Stage** - момент обработки инструкции: Before (до) или After (после) изменений. События On schema conversion start/finish не имеют настройки этого параметра, так как это не имеет смысла.
- **Code** - код обработчика на Python.
- **Description** - комментарий к логике работы обработчика.



Технически расширение на Python реализовано в виде автоматически генерируемого py-модуля, который создается во временной директории ОС. Модуль содержит в себе import-инструкции базовых платформенных модулей-обертки и класс с именем расширения. В классе определены методы с именами событий и соответствующим кодом обработчика.

В случае, если обработчик на событие до изменений вернул True, то выполнение инструкции будет отменено. По умолчанию Python-метод возвращает None, что эквивалентно False.

Внутри тела метода-обработчика можно использовать все типы данных и функции, определенные в обёртке платформы `sbis-python` (все объекты, определенные в модуле `sbis`). Допускается выполнять import-инструкции для любых py-модулей, которые находятся в корнях БЛ-модулей, входящих в состав набора разворачиваемых сервисов.

При подключении внешних модулей необходимо учесть изменение поведения платформенных Python-функций:

- функция `sbis.GetConnectedDatabase()` - вернет подключение к текущей конвертируемой БД.
- функции `sbis.SqlQuery(...)`, `sbis.SqlQueryRecord(...)`, `sbis.SqlQueryScalar(...)`, `sbis.SqlQueryRow(...)` - будут выполнены на текущей конвертируемой БД.

Параметры в сигнатуре обработчика:

- `context` - контекст инструкции. Тип: dict.
- `source_schema` - исходное состояние объектов схемы БД. Тип: класс `DBSchema`.

- `target_schema` - целевое состояние объектов схемы БД, сгенерированное по метаданным проекта. Тип: класс `DBSchema`.

Методы класса `DBSchema`:

- `def TableExists(table_name) -> bool` - проверка существования таблицы в данном объекте `DBSchema`.
- `def TableFieldExists(table_name, field_name) -> bool` - проверка существования поля в таблице объекта `DBSchema`.
- `def TableIndexExists( table_name, index_name ) -> bool` - проверка существования индекса на таблице объекта `DBSchema`.

Пример обработчика на Python

Пример кода обработчика инструкции на Python:

```

1  # подключение доп. модулей, чтобы не дублировать кодimport
   my_module_from_bl# вызов логики из стороннего
   модуляmy_module_from_bl.do_something()# проверка схемы БД - позволяет
   не выполнять тяжелый SQL-запросif
   source_schema.TableExists('my_log_table'):
2      # пример выполнения параметризованного пользовательского запроса#
   конвертируемой БД,# путь поиска инициализирован текущей конвертируемой
   схемойSqlQuery("""insert into my_log_table(log_fld) values ($1);""",
3      context["schema"])# установка пользовательской переменной
   в контекстcontext["my_state_flag"] = True# статус обработки
   инструкции: если инструкция имеет отношение к таблице#
   _00000003.my_table, то ее выполнение необходимо пропустить (выражение#
   представляет из себя компактную форму "if condition: return
   True")return context["schema"] == "_00000003" and context["table"] =
   'my_table'

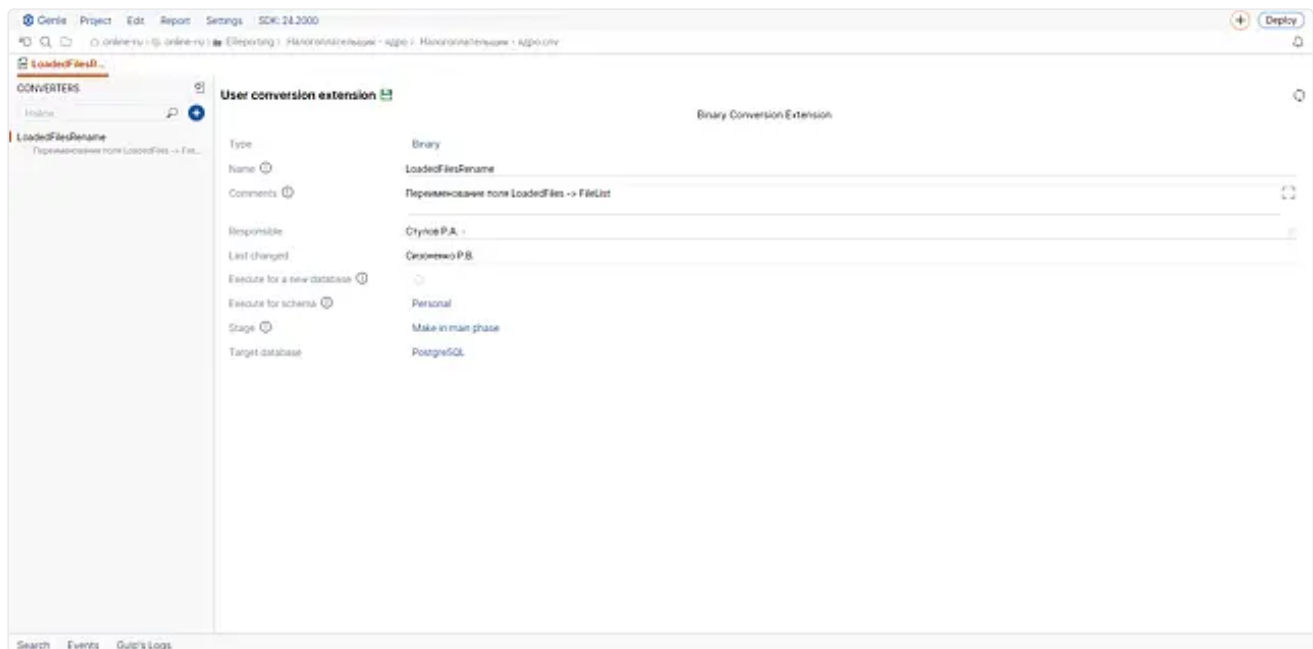
```

Пользовательское расширение конвертации на C++ (Binary-расширение)

Поскольку данный вид расширений полностью реализуется в C++ у него отсутствует дополнительный параметр **Callbacks**.

Технически Binary-расширение поставляется в скомпилированном бинарном коде разделяемого модуля и на момент конвертации разделяемый модуль должен быть загружен в память процесса **Конвертера БД**. Запись в контейнере `*.cnv` необходима для связи общих параметров расширения и реализации на C++. Если на момент конвертации реализация расширения с именем, указанным в метаданных, не существует, то процесс завершится с сообщением об ошибке.

Реализация расширения в C++ коде представляет из себя класс с произвольным именем, который публично наследуется от абстрактного класса `IBinaryConversionExtension`, определенного в заголовочном файле `sbis-db-converter/conversion_extension/binary_conversion_extension.hpp`.



Класс должен определить статический метод Name, который возвращает имя расширения в метаданных и реализовать виртуальный метод OnEvent, который реализует обработку необходимых событий.

Все поля класса представляют собой пользовательский контекст и являются потокобезопасными. Обращение к ним синхронизировать не требуется, так как каждый поток конвертации схемы имеет свою копию полей класса. В SQL и Python расширениях аналогичное поведение достигается созданием пользовательских полей в параметре context.

После определения реализации расширения ее необходимо зарегистрировать, при помощи макроса SBIS_REGISTER_BINARY_CONVERSION_EXTENSION.

Сигнатура OnEvent:

- ConversionEvent const event - структура-дескриптор события. Поля дескриптора:
 - ConversionActionStage mStage - момент вызова, enum со значениями: BEFORE (до) AFTER (после) изменений.
 - DDLAction mAction - DDL-тип действия, enum со значениями: CREATE, DROP, ALTER.
 - DBObject mObject - тип объекта БД над которым выполняется действие, enum со значениями: SCHEMA, STORED_PROCEDURE, SEQUENCE, VIEW, TABLE, COLUMN, CONSTRAINT_*, INDEX, TRIGGER. Состояние SCHEMA + BEFORE/AFTER является аналогом событий On schema conversion start/finish.
- OperationEnvironment const& context - контекст инструкции.
- DBSchema const& source - исходное состояние объектов схемы БД.
- DBSchema const& target - целевое состояние объектов схемы БД, сгенерированное по метаданным проекта.
- ConversionEventStatus& result - результат обработки инструкции, enum со значениями:
 - PASS - разрешить выполнение инструкции,
 - SKIP - запретить выполнение инструкции.

Пример обработчика на C++

Пример реализации бинарного расширения на C++:

```

1  #include <sbis-db-
    converter/conversion_extension/binary_conversion_extension.hpp>#include
    <sbis-db-converter/utils/operation_environment.hpp>#include <sbis-db-
    converter/utils/obj_utils.hpp>#include <sbis-
    dbi/misc/sql_query.hpp>#include <sbis-lib/utils/format.hpp>#include
    <sbis-lib/types/string_view.hpp>struct TestBinaryConversionExtension :
    IBinaryConversionExtension
2  {
3
4      bool mMyStateFlag{false};
5
6      static StringView Name(){
```

```

7         // возвращаем имя, которое указали при создании расширения в
контейнере// ресурсовreturn L"ТестовоеБинарноеРасширение"_sv;
8     }
9
10    // переопределяем pure-virtual методvirtual void OnEvent(
ConversionEvent const event,
11        OperationEnvironment const& context,
12        DBSchema const& /*source*/,
13        DBSchema const& target,
14        ConversionEventStatus& result ) override final
15    {
16        // селектор типа объектаswitch( event.mObject )
17        {
18            case DBOBJECT::SCHEMA:
19                // аналог события OnSchemaConversionStart/Finish// Пример
запроса, SqlQuery будет перенаправлен на текущую// конвертируемую БД,
путь поиска инициализирован конвертируемой// схемойSqlQuery(
sbis::Format( L"create table \"On%1%SchemaConversionFlagTable\"();"_sv,
20                    event.mStage ==
ConversionActionStage::AFTER ?
21                        L"After"_sv : L"Before"_sv ) );
22                break;
23            case DBOBJECT::CONSTRAINT_PK:
24                // Отмена действий над ограничением первичного ключа на
таблице// test_binaryext_tableif( L"test_binaryext_table"_sv ==
context[ EXTENSION_TABLE ] )
25                result = ConversionEventStatus::SKIP;
26                break;
27            case DBOBJECT::SEQUENCE:
28                // Отмена действий над последовательностью с именем//
test_binaryext_table_@test_binaryext_table_seqif(
L"test_binaryext_table_@test_binaryext_table_seq"_sv == context[
EXTENSION_SEQUENCE ] )
29                result = ConversionEventStatus::SKIP;
30                break;
31            case DBOBJECT::INDEX:
32                // Отмена действий над индексом itest_binaryext_table-bidx//
на таблице test_binaryext_tableif( L"test_binaryext_table"_sv ==
context[ EXTENSION_TABLE ] &&
33                L"itest_binaryext_table-bidx"_sv == context[
EXTENSION_INDEX ] )
34                result = ConversionEventStatus::SKIP;
35
36                // пример установки пользовательского флага в зависимости от//
события и состояния целевой схемыmMyStateFlag = TryFindTableInSchema(
L"TestTable", target );
37                break;
38        }
39        // все остальное пропускаем без изменений}
40    };// регистрация реализации в
синглтонеSBIS_REGISTER_BINARY_CONVERSION_EXTENSION(
TestBinaryConversionExtension );

```

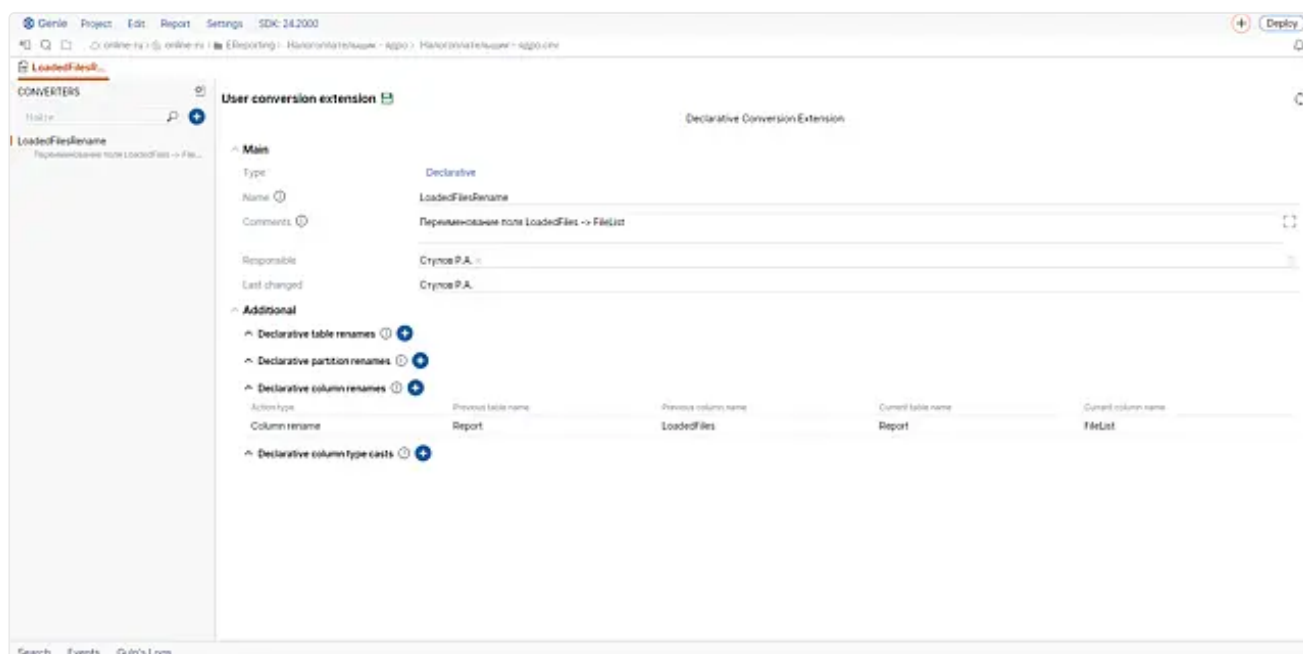
Декларативное пользовательское расширение

Декларативные расширения позволяют разработчику описать изменения в проекте без написания кода с помощью типовых декларативных действий. Расширения данного типа запускаются также, как и обычные расширения - только один раз с фиксированными параметрами: только на существующей БД и только на основной фазе конвертации. У них отсутствуют дополнительные параметры. Технически Declarative-расширения внутри конвертера БД преобразуются в набор автоматических патчей для первичного плана конвертации и применяются до начала процесса конвертации.

Данный тип расширений был разработан для часто повторяющихся типовых ситуаций, когда обычные типы расширений слишком сложны в реализации. Например, чтобы переименовать таблицу, секцию или поле таблицы потребуются заблокировать создание и удаление множества зависимых объектов БД, так как при построении плана конвертации невозможно автоматически установить соответствие между двумя объектами данных типов без дополнительных "подсказок", что приведет к генерации инструкций по удалению старых объектов и созданию новых с потерей данных. Также от пользователя потребуются учесть локализованность таблиц и необходимость переименования всех служебных объектов, таких как log- и blob- таблицы, таблицы секций, поля связи Categorization/On Demand, первичные ключи расширяющих таблиц и т. д.

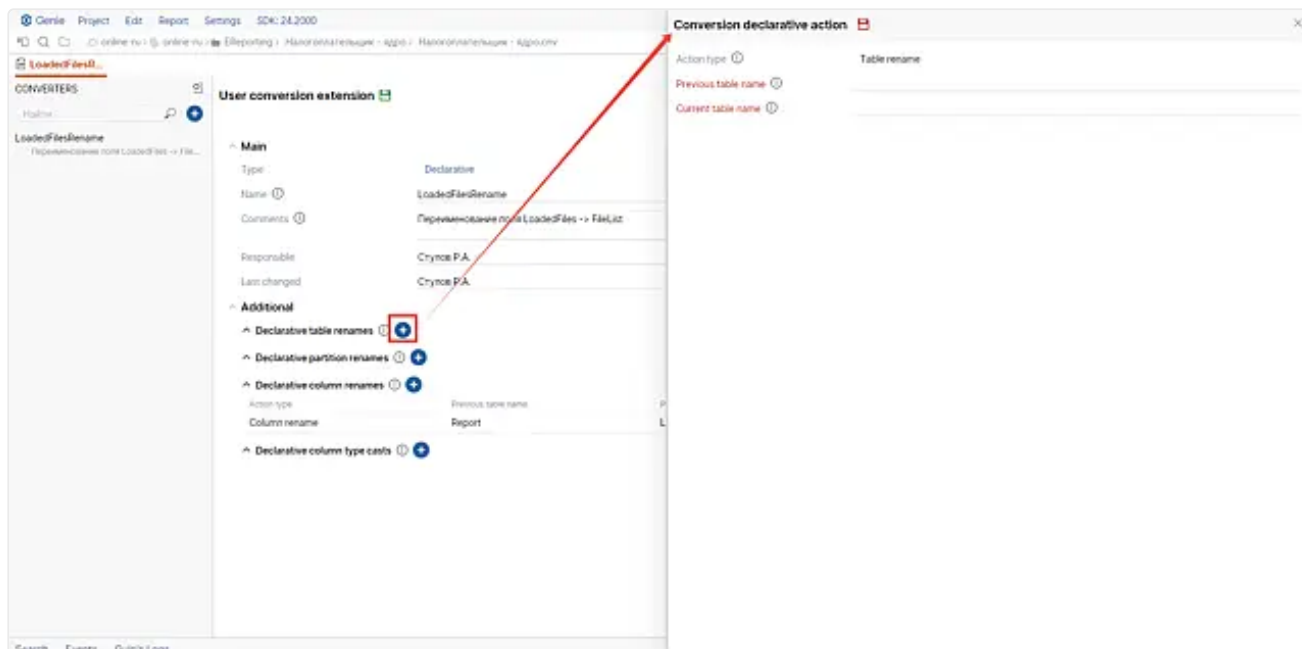
Интерфейс описания декларативных расширений состоит из четырех плоских списков, каждый из которых содержит декларативные описания изменений, сгруппированные по типу декларативного действия:

- **Declarative table renames** - переименование таблиц.
- **Declarative partition renames** - переименование секций таблиц.
- **Declarative column renames** - переименование колонок таблиц.
- **Declarative type change** - смена типа колонок.



Декларативное переименование таблицы

При создании декларативного действия переименования таблицы необходимо указать предыдущее имя таблицы в поле **Previous table name** и текущее имя, выбрав вариант из списка имен таблиц проекта, в поле **Current table name**.



Пример №1 - Типичное переименование таблицы

1. Таблица А описана в проекте и развернута в БД.
2. Прикладной разработчик переименовывает в метаданных проекта таблицу А в В.
3. Прикладной разработчик создает декларативное расширение.
4. Прикладной разработчик добавляет декларативное действие переименования таблицы, где в поле **Previous table name** записывает значение А, а в поле **Current table name** выбирает В из списка таблиц проекта.
5. В плане конвертации будет создана запись по переименованию А в В, при этом, если в БД существует таблица В, то она будет удалена.

Пример №2 - Конфликтное переименование таблицы

1. Таблицы А и Б описаны в проекте и развернуты в БД.
2. Прикладной разработчик переименовывает в метаданных проекта таблицу А в Б и Б в А.
3. Прикладной разработчик создает декларативное расширение.
4. Прикладной разработчик добавляет декларативное действие переименования таблицы, где в поле **Previous table name** записывает значение А, а в поле **Current table name** выбирает В из списка таблиц проекта.
5. Прикладной разработчик добавляет декларативное действие переименования таблицы, где в поле **Previous table name** записывает значение В, а в поле **Current table name** выбирает А из списка таблиц проекта.
6. В плане конвертации будет создана запись по переименованию А в В и В в А, конфликт имен разрешен промежуточным переименованием таблиц специальным образом.

Пример №3 - Неоднозначное переименование таблицы

1. Таблица А описана в проекте и развернута в БД.
2. Прикладной разработчик переименовывает в метаданных проекта таблицу А в В.
3. Прикладной разработчик создает декларативное расширение.
4. Прикладной разработчик добавляет декларативное действие переименования таблицы, где в поле **Previous table name** записывает значение А, а в поле **Current table name** выбирает В из списка таблиц проекта.
5. Прикладной разработчик добавляет декларативное действие переименования таблицы, где в поле **Previous table name** записывает значение А, а в поле **Current table name** выбирает С из списка таблиц проекта.
6. При сохранении метаданных проекта произойдет ошибка неоднозначного переименования.



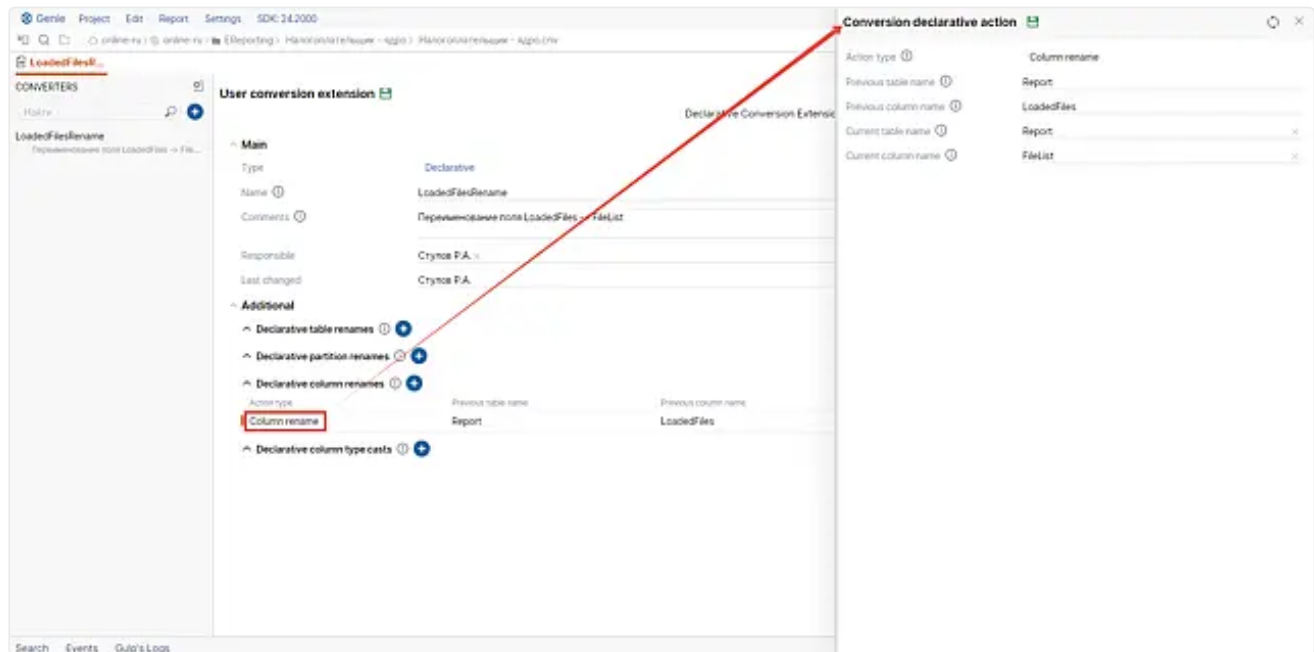
Валидация некорректных данных

Декларативные действия - это вспомогательный инструмент, облегчающий жизнь разработчика. Валидация не всегда может отследить ввод некорректных данных, что может приводить к ошибкам или неожиданным результатам во время конвертации. Предполагается, что пользователь компетентен и перед тем как запустить свое расширение конвертации проверил все на локальной копии БД.

Декларативное переименование колонки таблицы

При создании декларативного действия переименования колонки таблицы необходимо указать предыдущее имя таблицы в поле **Previous table name** и текущее имя, выбрав вариант из списка имен таблиц проекта, в поле **Current table name**.

Далее необходимо указать предыдущее имя колонки в поле **Previous column name** и текущее имя колонки, выбрав вариант из списка колонок таблицы, в поле **Current column name**.



Если была переименована только колонка, то имена **Previous table name** и **Current table name** должны совпадать.

Если была переименована только таблица, но не колонки, то создавать правила переименования колонок **не нужно**, так как одноименные колонки с приводимыми типами будут преобразованы в автоматическом режиме.

Пример №1 - Типичное переименование колонки

1. Колонка A описана в проекте и развернута в БД.
2. Прикладной разработчик переименовывает в метаданных проекта колонку A.C в A.C1.
3. Прикладной разработчик создает декларативное расширение.
4. Прикладной разработчик добавляет декларативное действие переименования колонки, где в поле **Previous table name** записывает значение A, в поле **Previous column name** записывает значение C, в поле **Current table name** выбирает A из списка таблиц проекта, в поле **Current column name** выбирает C1 из списка колонок таблицы A.
5. В плане конвертации будет создана запись по переименованию колонки A.C в A.C1.

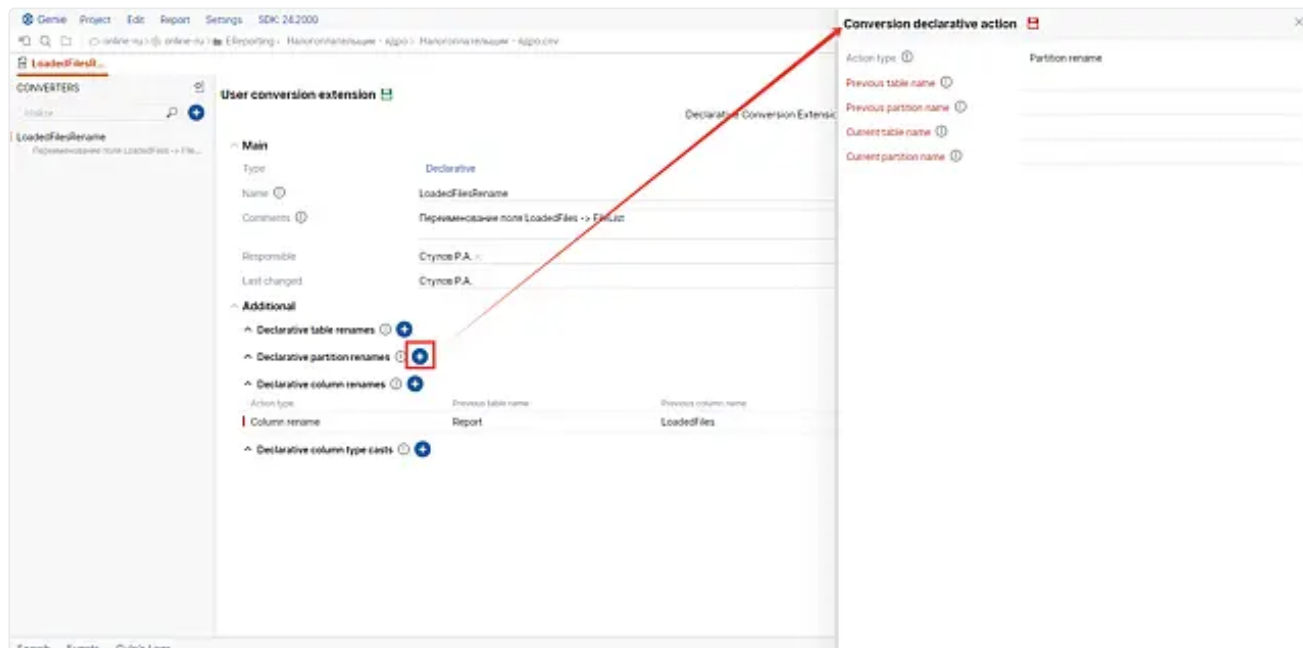
Пример №2 - Полное переименование колонки

1. Таблица A.C описана в проекте и развернута в БД.
2. Прикладной разработчик переименовывает в метаданных проекта таблицу A в B и колонку A.C в B.C1.
3. Прикладной разработчик создает декларативное расширение.
4. Прикладной разработчик добавляет декларативное действие переименования таблицы, где в поле **Previous table name** записывает значение A, а в поле **Current table name** выбирает B из списка таблиц проекта.
5. Прикладной разработчик добавляет декларативное действие переименования колонки, где в поле **Previous table name** записывает значение A, в поле **Previous column name** записывает значение C, в поле **Current table name** выбирает B из списка таблиц проекта, в поле **Current column name** выбирает C1 из списка колонок таблицы B.
6. В плане конвертации будет создана запись по переименованию таблицы A в B и колонки A.C в B.C1.

Примеры конфликтного и неоднозначного переименования аналогичны примерам для таблиц

Декларативное переименование секции таблицы

При создании декларативного действия переименования секции таблицы необходимо указать предыдущее имя таблицы в поле **Previous table name** и текущее имя, выбрав вариант из списка имен таблиц проекта, в поле **Current table name**. Далее необходимо указать предыдущее имя секции в поле **Previous partition name** и текущее имя секции, выбрав вариант из списка секций таблицы, в поле **Current partition name**.



Если была переименована только секция, то имена **Previous table name** и **Current table name** должны совпадать.

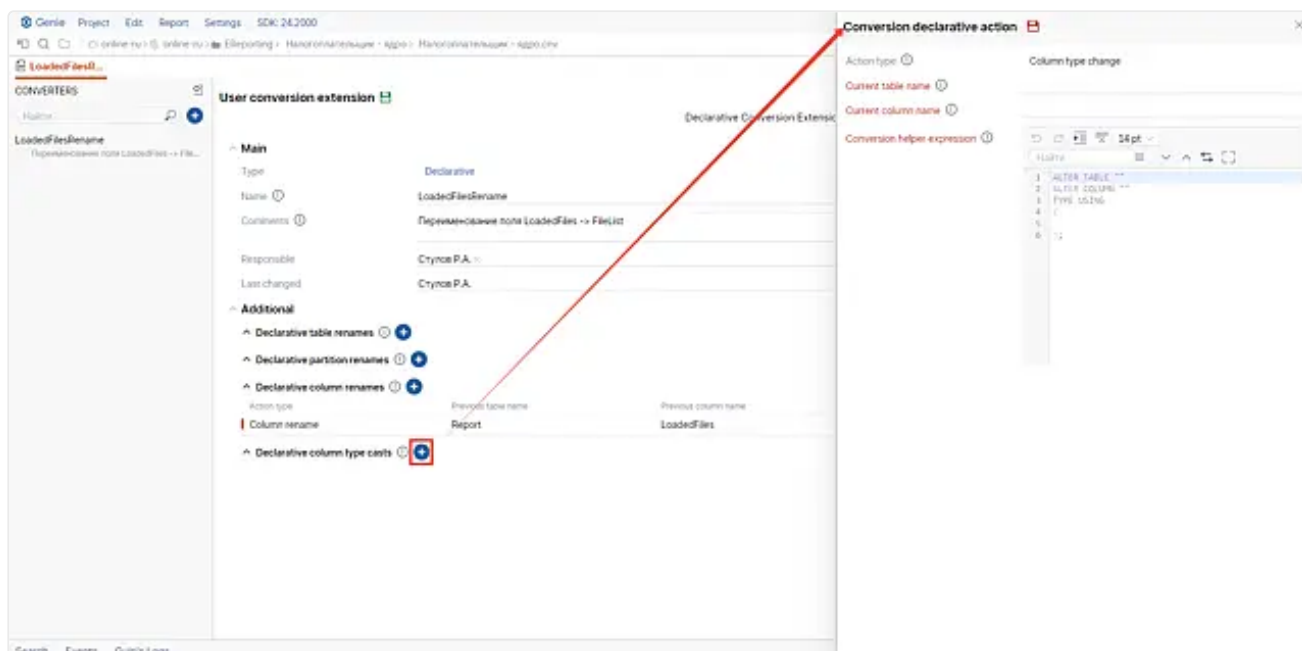
Если была переименована только таблица, но ни одна из ее секций, то создавать правила переименования секций **не нужно**, так как одноименные секции будут преобразованы в автоматическом режиме.

Примеры аналогичны примерам переименования колонки.

Декларативная смена типа колонки

Данный тип расширений может быть полезен когда при переименовании таблицы/колонки меняется еще и тип данных, который несовместим с предыдущим. В этом случае конвертер сгенерирует инструкции удаления и создания колонки, несмотря на декларативные предписания, так как преобразование типа не задано. Для разрешения подобных ситуаций в набор декларативных действий было добавлено декларативное действие по преобразованию типа.

При создании декларативного действия смены типа колонки таблицы необходимо указать текущее имя таблицы, выбрав вариант из списка имен таблиц проекта, в поле **Current table name**. Далее необходимо указать текущее имя колонки, выбрав вариант из списка колонок таблицы, в поле **Current column name**. Далее необходимо указать выражение для преобразования типа колонки, которое будет использовано в SQL-инструкции ALTER COLUMN.



Расширения данного типа имеют смысл только для СУБД PostgreSQL.

Пример №1 - Типичное приведение типа

1. Колонка A.C с типом Decimal описана в проекте и развернута в БД.
2. Прикладной разработчик меняет тип колонки Decimal в метаданных проекта на String.
3. Прикладной разработчик создает декларативное расширение.
4. Прикладной разработчик добавляет декларативное действие приведения типа колонки, где в поле **Current table name** выбирает A из списка таблиц проекта, в поле **Current column name** выбирает C из списка колонок таблицы A. Далее в поле **Conversion helper expression** вводит выражение "C"::text || ' %'.
5. При конвертации будет добавлена инструкция ALTER COLUMN с секцией USING ("C"::text || ' %').

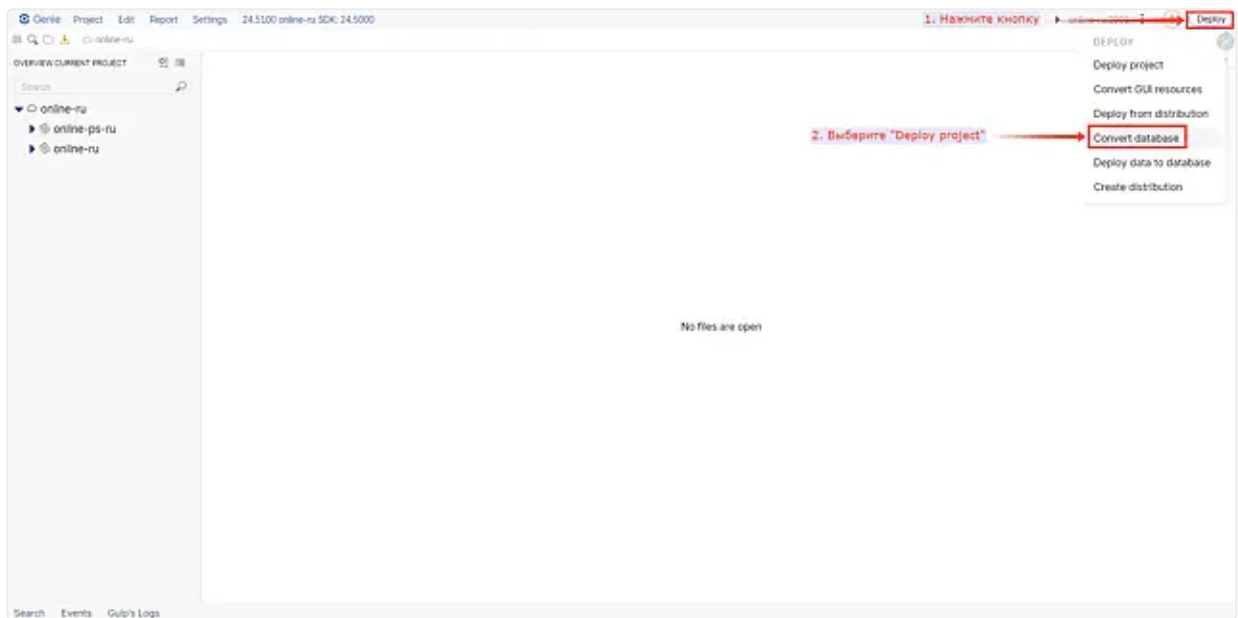
Пример №2 - Комплексное приведение типа

1. Колонка A.C с типом String описана в проекте и развернута в БД.
2. Прикладной разработчик переименовывает в метаданных проекта таблицу A в B, меняет тип колонки C на DateTime.
3. Прикладной разработчик создает декларативное расширение.
4. Прикладной разработчик добавляет декларативное действие переименования таблицы, где в поле **Previous table name** записывает значение A, а в поле **Current table name** выбирает B из списка таблиц проекта.
5. Прикладной разработчик добавляет декларативное действие приведения типа колонки, где в поле **Current table name** выбирает B из списка таблиц проекта, в поле **Current column name** выбирает C из списка колонок таблицы B. Далее в поле **Conversion helper expression** вводит выражение to_timestamp("C", '!DD~Mon~YYYY::HH12:MI:SS!').
6. При конвертации будет добавлена инструкция ALTER COLUMN с секцией USING (to_timestamp("C", '!DD~Mon~YYYY::HH12:MI:SS!')).

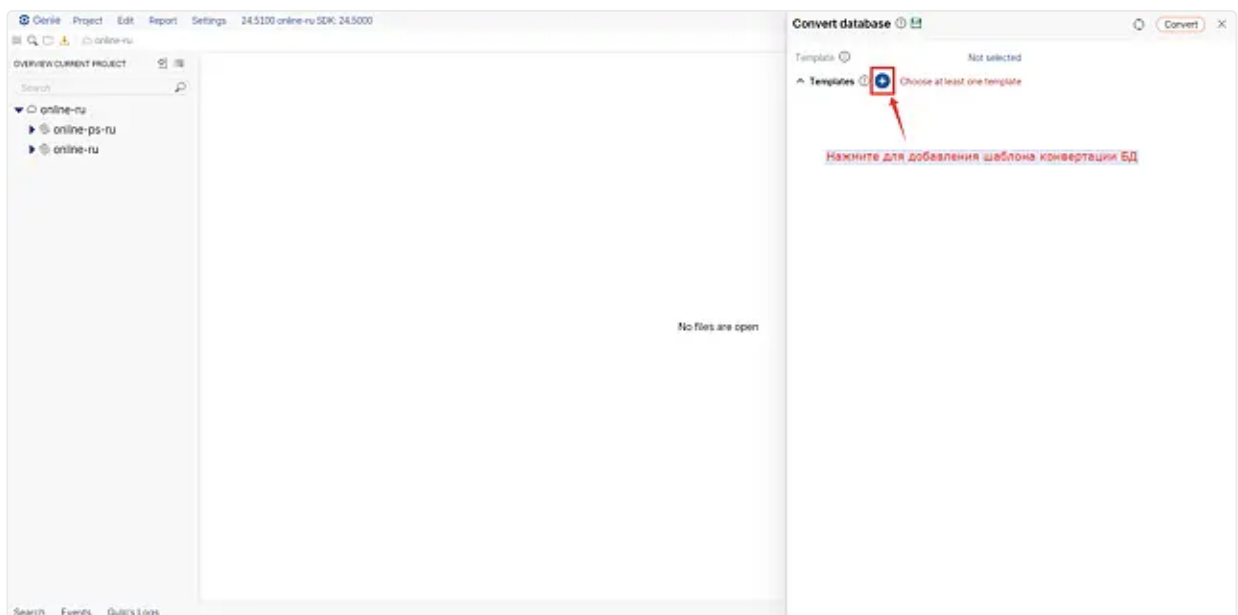
Проверка и отладка расширения конвертера БД

Отладка доступна только для серверных расширений SQL и Python. Декларативные расширения реализуются платформой и их отладка недоступна.

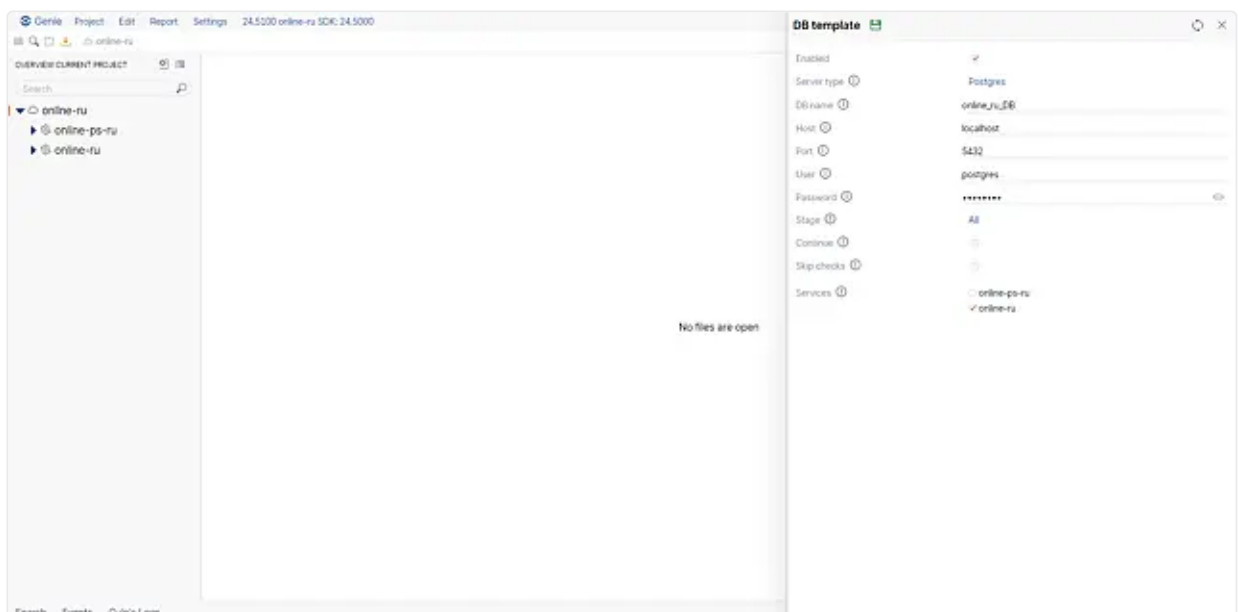
1. Подготовьте данные для проверки на локальном стенде. Загрузите тестовые данные в БД любым удобным способом.
2. Запустите конвертацию с помощью утилиты [Convert database:](#)
 - В Genie нажмите кнопку Deploy → Convert database:



- Добавьте шаблон конвертации:



- Выберите базу для конвертации:



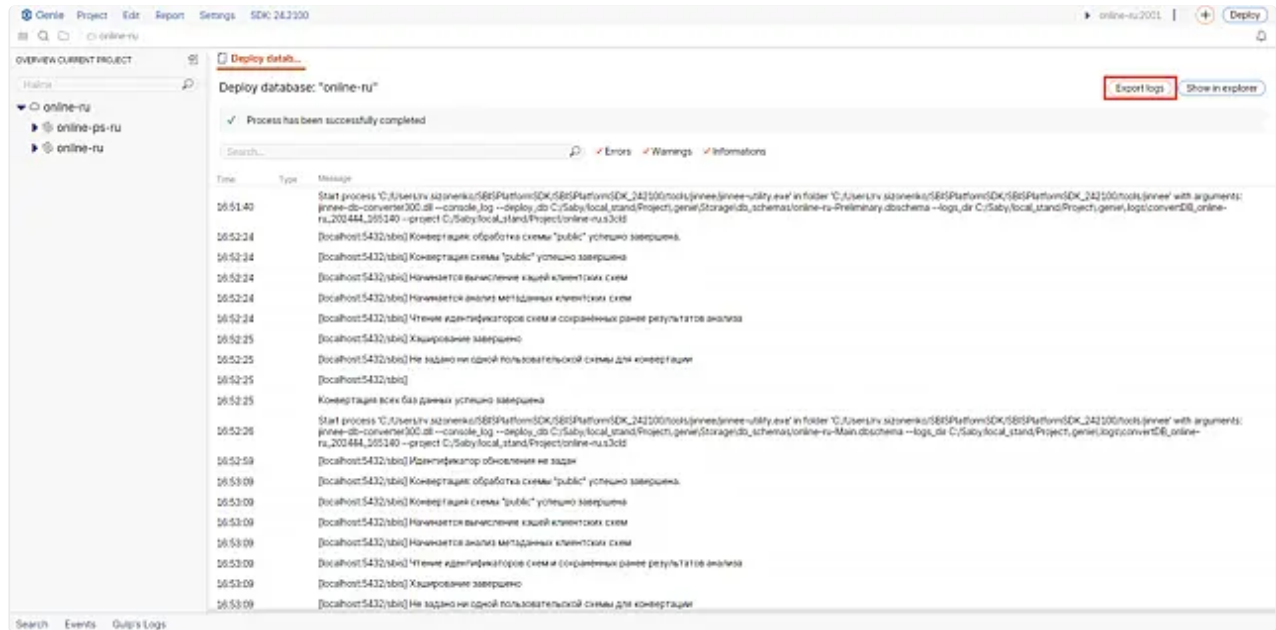
- Для запуска процесса конвертации нажмите кнопку Convert.



Если в базе не будет ни одной схемы и указана типовая настройка Personal, то расширение будет

зарегистрировано, но не выполнено.

3. Нажмите кнопку "Export logs" для скачивания логов.



В логах указаны настройки конвертера (Conversion settings), в частности, фазы конвертации (Stage).

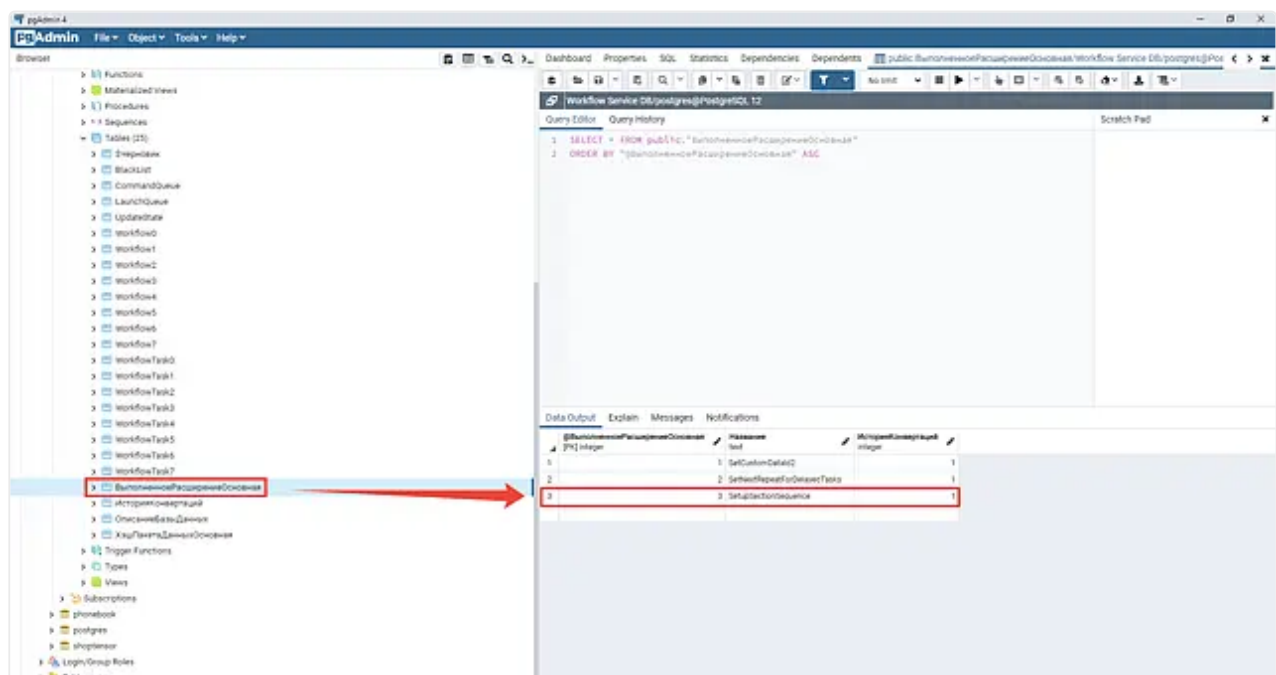
```
=== Conversion settings ===
Update identifier: not set
Stage: Main
Continue conversion mode: no
Partial mode: no
Non-blocking (lightweight) mode: no
Forced execution mode: no
Invalidate schema hashes: no
Conversion thread count: 6
Schemas to skip: none
```

Подробно о фазах конвертации описано [здесь](#).

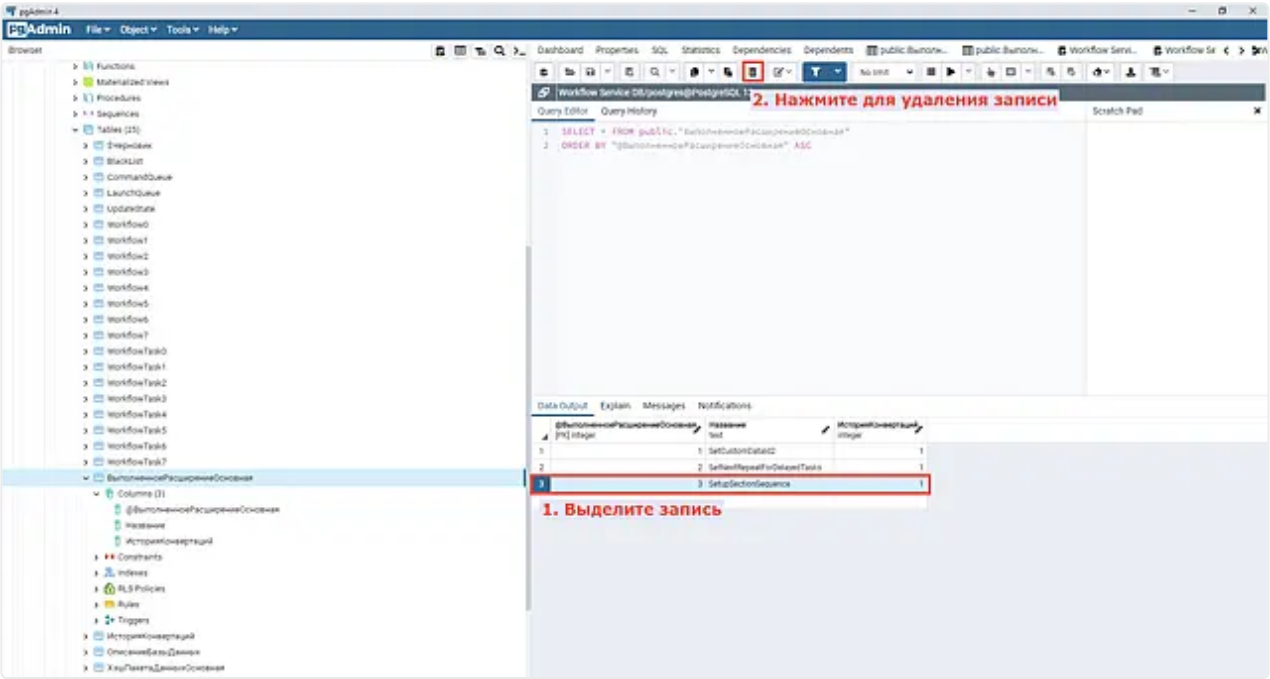
Найдите отладку расширения конвертера в логе по имени расширения и фразе "Вызов метода-расширения".

```
COMMENT ON SEQUENCE public."HashPaketaDannishOsnovnaya_@HashPaketaDannishOsnovnaya_seq"
IS 'sbls3_object';
16:21:14.065 [8792][14056] 49500160 INFO [sql][finish] [exec period] 0
16:21:14.066 [8792][14056] 49082368 INFO Вызов метода-расширения "SetupSectionSequence" "На завершение конвертации схемы" с контекстом ("cnv_stage"=>"main", "schema"=>"public").
16:21:14.066 [8792][14056] 49082368 INFO [sql][start] [localhost:5432/workflow service 10][public][0]
```

4. После конвертации проверьте БД.



При повторном запуске конвертации на этой же БД, расширение **запускаться не будет**. Для того, чтобы **запустить конвертацию еще раз**, удалите запись из таблицы **"ВыполненноеРасширениеОсновная"**.



либо поменяйте имя расширения в Genie.

5. Если результат не устраивает, запустите конвертер повторно.

Data Output Explain Messages Notifications			
	@ВыполненноеРасширениеОсновная [PK] integer	Название text	ИсторияКонвертаций integer
1		1 SetCustomDataId2	1
2		2 SetNextRepeatForDelayedTasks	1
3		3 SetupSectionSequence	1



У каждой схемы своя история расширений. Все расширения хранятся в таблице **"ВыполненноеРасширениеОсновная"**.

Также вы можете повторить конвертацию, удалив базу и развернув ее из старого дистрибутива. После этого заново сконвертируйте БД под новый дистрибутив с внесенными правками.

Помимо SQL и Python существуют расширения конвертера на C++, они применяются в мобильных и десктопных приложениях. Схема регистрации аналогична описанной выше, но отладка осуществляется отладчиком C++.

Установите в теле класса "Поставить точку останова" и запустите приложение под отладчиком. После запуска придет управление в расширение конвертера.